A WORKED EXAMPLE

I could not talk to half my colleagues.

So I built a thing that fixes it — in a way that was not available to me two years ago. This talk is about **how** I built it, more than about what it does.

`Live captions for meetings · built in days, not months · costs zero to run`

THE PROBLEM

English is our working language. On paper.

- Written communication among devs is in English, and that works.
- Spoken communication is **where it breaks**. Not everyone is comfortable in English at speaking speed.
- I speak almost no Portuguese. In a meeting I lose the questions, not the slides.
- Teams has live captions. **Translated** captions need Premium, and everyone would need it.

The shape of the problem

A presentation is a **one-to-many** broadcast with a **many-to-one** Q&A tail. Those two directions have completely different requirements — and that turned out to be the single most important thing to notice.

Me → room	`latency critical`
Room → me	`accuracy critical`

WHY NOW

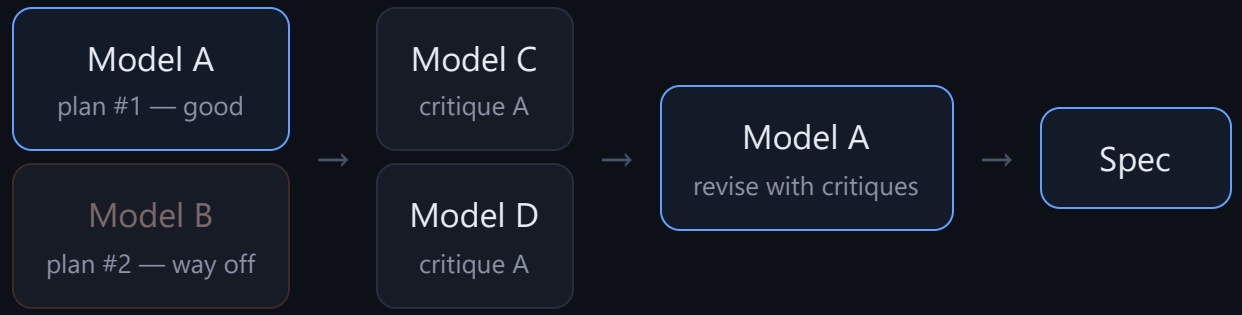
The same idea, two years apart

	THEN	NOW
Speech recognition	Paid cloud service, API key, per-hour billing	Built into the browser. Free, unlimited.
Translation	Paid API, network round-trip per phrase	Runs **on the device**. No network, no key, no quota.
Language detection	Specialist model, own tuning, own failure modes	One sentence in a prompt.
Building it	Weeks of a team. Would never be approved for one person's problem.	Days of one person. Approval not required.

> The interesting shift is the last row. **The economics changed before the technology did.** Problems that affect one person were never worth a project. Now they are worth an afternoon — so they get solved.

METHOD

I did not ask one model. I convened a council.



- Ask **2–4 models the same question**, independently. Cheap, parallel.
- Pick the best plan. Here the first one was already good and the second was clearly wrong — **that comparison is the value**, not the average.
- Hand the winner to the **others as critics**, not as authors.
- Feed the critiques **back to the original author**.

This has a name now

Karpathy's **LLM Council** — parallel query, peer review, synthesis. The variant I used is the **adversarial council**: reviewers are told to find flaws, not to agree.

Why it works: a single model's blind spot becomes the final answer. A critic with a different blind spot does not share it.

EVIDENCE

What the critics actually changed

FIRST PLAN SAID	FINAL SAYS	WHY IT FLIPPED
Azure Speech as the engine	No paid services at all	Needs a card even for the free tier. Kills adoption by colleagues.
Accept PPTX, convert with LibreOffice	PDF only	Removed a server, an OOXML parser, and a whole class of layout bugs.
Hand-written audio resampler + unit test	One constructor argument	The browser already resamples. A day of work that did not need doing.
Languages fixed in code	A profile object	"Your colleagues have other languages." I had not thought of that at all.

> Every one of these **removed** work. The critics did not add features — they deleted plans. That is the highest-value thing a reviewer can do, and it is exactly what a single enthusiastic model will not do to itself.

METHOD

The spec is the artifact. Code is downstream.

- I wrote and rewrote a **specification**, not prompts. Requirements, acceptance criteria, explicit non-goals.
- Every reversal from the previous slide is **recorded in it with the reason**. Six weeks from now I will not re-litigate Azure.
- The industry calls this **spec-driven development**, and 2026 is the year every agent tool shipped a flavour of it.
- The stated reason matches my experience: agents write code well and **guess intent badly**. Drift is the failure mode, not syntax.

What made the difference

Non-goals. "No PPTX. No accounts. No server." Written down, they stop coming back.

Acceptance criteria as tests. "Transcripts contain no phrase in the other language" is checkable. "Good quality" is not.

A decision log. The spec ends with a table of what was reversed and why — the most reread page.

METHOD

Measure before you build. One page of code.

Before writing the app I wrote a single HTML file that tested every assumption on the actual laptop. It took minutes and it changed the architecture.

147 ms

Browser speech recognition, end of sentence to final text. I had budgeted **1000 ms**.

~600

Requests to a cloud model for a 2-hour talk, at my real speaking rate. Almost certainly over the free daily quota.

0

Drivers, servers and accounts needed — all three risks I had planned mitigations for turned out not to exist.

> Both numbers pointed the same way and **reversed my default**. I had planned to use the cloud model for my own speech. Measurement said: use the browser, keep the cloud for the audience. Twenty times faster, and free.

THE PRODUCT · LIVE DEMO

Two windows. Only one is shared.

shared into Teams

slide + annotations drawn live

Executamos o Kubernetes em três regiões.

Captions live **inside** the shared window, so they reach the audience as part of the picture. No bot, no Premium, nothing for them to install.

never shared

two full transcripts

Here I want to dwell on…

Aqui quero deter-me em…

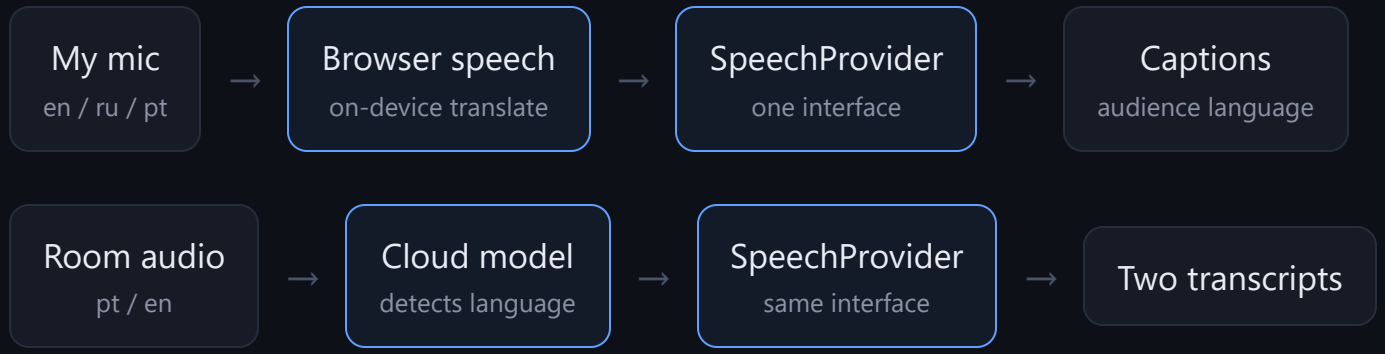
Sorry, a quick question…

Desculpem, uma pergunta…

The log, the originals, my notes — **mine only**.

ARCHITECTURE

Two channels, two engines, one contract



LAYER	CHOICE	WHY THAT ONE
Slides	`pdf.js in canvas`	Only reliable way to render a print-accurate slide
Window sync	`BroadcastChannel`	Native, no server, works between windows
Annotations	`SVG over canvas`	The PDF is never modified. Browser does hit-testing.
Storage	`IndexedDB + localStorage`	Log survives an accidental F5; nothing leaves the machine

THE DECISION THAT SHAPED EVERYTHING

Let the user choose, then let the choice pick the engine

	PIN – I SAY WHICH LANGUAGE	AUTO – THE MODEL DETECTS IT
Caption delay	~0.15 s	~3 s
Text as you speak	yes	no — whole phrase at once
Daily limits	none	shared quota
API key	not needed	required
I must remember	press a key before switching	nothing

Mic pinned, room auto. I know which language I am about to speak. I cannot know which language the next question comes in.

The setting is not cosmetic — it **selects the engine**. The app never asks "which provider?", it asks a question the user can actually answer, and derives the rest.

THE TOOLBOX

Which fashionable technique, and when

TECHNIQUE	WHAT IT ACTUALLY IS	WORTH IT WHEN	OVERHEAD WHEN
✓ **LLM council**	Several models answer, then critique each other	Architecture and irreversible choices	Routine code you can just read
✓ **Spec-driven**	A versioned spec is the source of truth; code is generated from it	Anything you will still be changing next month	A script you run once
✓ **Dev harness**	Mock provider + a demo route replaying a recorded script	Input is a live human, hardware or a paid API	Pure functions — just call them
✓ **Probe first**	Throwaway page measuring your assumptions on the real machine	You are about to design around a guessed number	The number is documented and stable
Evals	A scored test set that catches quality regressions	Model output is the product and it ships repeatedly	One-off tool, human in the loop every run
Voice assistant	Speech-to-speech session over WebRTC, sub-second	Hands and eyes are busy — driving, presenting, surgery	Typing is faster and quieter, which is most of the time

✓ marks what this project actually used. The mock provider is the one I would defend hardest: **without it, every UI change needs a live human talking into a microphone.**

HONEST OMISSION · WHAT CHANGES NEXT

Agent orchestration: I did not use it

The 2026 patterns are settled — supervisor, pipeline, parallel fan-out, router, hierarchical, evaluator-optimizer. I used none of them, and I think that was right.

- **Why not:** one app, one developer, work that is mostly sequential. Orchestration buys parallelism I did not have to spend.
- **When it pays:** a mechanical change across many files — migrations, audits, sweeps. Fan-out under a supervisor.
- **When it pays here, later:** the review pass. Several critics on different dimensions, running at once, is exactly my council — just automated instead of copy-pasted.

What improves as models improve

- On-device translation gets better → the free path stops being the compromise and becomes the default.
- Streaming speech models mature → detection and low latency stop being a trade-off, and my central design decision **evaporates**.
- Cheaper audio tokens → the quota argument disappears.
- Longer context → the whole meeting summarised, not just transcribed.

> The provider interface exists so that **none of these require rewriting the app** — only adding a file.